

GROUP BY y funciones de agregación en SQL

Hasta ahora todas las consultas devuelven filas individuales: un cliente, un producto, un pedido. Pero las preguntas de negocio reales no funcionan así. Nadie pregunta "¿cuántos pedidos tiene el cliente 81?". Preguntan "¿cuánto vendemos por país?", "¿cuál es el ticket promedio por sucursal?" o "¿cuántos clientes tiene cada categoría?". Esas preguntas se responden resumiendo millones de filas en unos pocos números. Para eso existen **GROUP BY** y las funciones de agregación.

Qué hace GROUP BY

GROUP BY agrupa las filas que tienen el mismo valor en una columna y permite aplicar una operación de resumen sobre cada grupo. En lugar de devolver una fila por registro, devuelve una fila por grupo.

Antes de ver ejemplos, la regla más importante:

Toda columna del **SELECT que no sea una función de agregación debe estar en el **GROUP BY**.**

Si una columna aparece en el **SELECT** pero no es una agregación y tampoco está en el **GROUP BY**, la consulta falla. No es una sugerencia: es una restricción que el motor aplica siempre. Más adelante vas a ver exactamente por qué tiene sentido.

Las funciones de agregación

COUNT — contar filas

```
SQL
SELECT COUNT(*) AS total_clientes
FROM clientes;
```

COUNT(*) cuenta todas las filas de la tabla. Si quieres contar solo las filas que tienen un valor en una columna específica (ignorando los nulos), usas el nombre de la columna en lugar del asterisco:

```
SQL
SELECT
    COUNT(*)          AS total_clientes,
    COUNT(telefono) AS clientes_con_telefono
FROM clientes;
```

Si hay clientes sin teléfono registrado, los dos números van a ser distintos. `COUNT(*)` los cuenta a todos; `COUNT(telefono)` solo los que tienen ese campo lleno.

SUM — sumar valores

```
SQL
SELECT SUM(total) AS total_ventas
FROM pedidos;
```

Suma todos los valores de una columna numérica. Útil para totales de ventas, ingresos acumulados, stock total.

AVG — calcular el promedio

```
SQL
SELECT AVG(precio) AS precio_promedio
FROM productos;
```

Calcula el promedio de todos los valores de la columna. Ideal para tickets promedio, precios medios por categoría o promedios de cualquier métrica numérica.

MIN y MAX — el valor más bajo y el más alto

```
SQL
SELECT
    MIN(precio) AS precio_minimo,
    MAX(precio) AS precio_maximo
FROM productos;
```

Útiles para encontrar rangos: el pedido más pequeño, la venta más alta, la fecha más antigua.

GROUP BY en acción: los ejemplos reales

Cantidad de pedidos por cliente

```
SQL
SELECT
    cliente_id,
    COUNT(*) AS cantidad_pedidos
FROM pedidos
GROUP BY cliente_id
ORDER BY cantidad_pedidos DESC
LIMIT 10;
```

Esto devuelve los diez clientes con más pedidos. Fíjate en la regla: `cliente_id` no es una función de agregación, así que va en el `GROUP BY`. `COUNT(*)` sí es una función de agregación, así que no necesita estar ahí.

Pedidos y ventas por mes

Aquí se combinan las funciones de fecha que ya conoces con las funciones de agregación:

```
SQL
SELECT

    EXTRACT(YEAR FROM fecha_pedido) AS anio,

    EXTRACT(MONTH FROM fecha_pedido) AS mes,

    COUNT(*)           AS cantidad_pedidos,

    SUM(total)         AS total_ventas

FROM pedidos

GROUP BY

    EXTRACT(YEAR FROM fecha_pedido),

    EXTRACT(MONTH FROM fecha_pedido)

ORDER BY anio, mes;
```

`EXTRACT(YEAR FROM fecha_pedido)` no es una función de agregación, es una transformación de fecha. Por eso va en el `GROUP BY`. El resultado es una fila por cada combinación de año y mes, con la cantidad de pedidos y el total vendido en ese período.

Atención con los nombres de columna: antes de escribir cualquier consulta, verifica en pgAdmin cómo se llama exactamente la columna en tu tabla. Si la tabla tiene `fecha_pedido` y tú escribes `fecha`, la consulta va a fallar. Expande la tabla en el navegador de objetos y confirma el nombre exacto.

Por qué la regla del GROUP BY existe

Imagina que agrupas pedidos por `cliente_id` y en el `SELECT` incluyes la columna `fecha_pedido` sin agregarla. El motor no sabe qué fecha mostrarte: ¿la del primer pedido del cliente, la del último, alguna aleatoria? No hay una respuesta correcta. Por eso la rechaza. Solo hay dos opciones válidas para una columna en un `SELECT` con `GROUP BY`: que sea el criterio de agrupación (y esté en el `GROUP BY`) o que sea el resultado de una función de agregación.

Para verlo en acción, prueba quitando deliberadamente una columna del `GROUP BY` que no sea una agregación. La consulta va a devolver un error. Vuelve a agregarla y el error desaparece. Ese ejercicio hace que la regla quede grabada.

Combinar varias funciones de agregación

No hay límite de funciones de agregación en una misma consulta. Puedes combinar todas las que necesites:

```
SQL
SELECT
    categoria_id,
    COUNT(*)      AS total_productos,
    AVG(precio)   AS precio_promedio,
    MIN(precio)   AS precio_minimo,
    MAX(precio)   AS precio_maximo
FROM productos
GROUP BY categoria_id
ORDER BY total_productos DESC;
```

Una sola consulta, cinco métricas, agrupadas por categoría. Ese es el poder de `GROUP BY`.

Referencia rápida

Función	Qué calcula	Ejemplo de uso
<code>COUNT(*)</code>	Cantidad de filas en el grupo	Número de pedidos por cliente
<code>COUNT(columna)</code>	Filas con valor no nulo en esa columna	Clientes que tienen teléfono
<code>SUM(columna)</code>	Suma de todos los valores	Total vendido por mes
<code>AVG(columna)</code>	Promedio de los valores	Ticket promedio por sucursal
<code>MIN(columna)</code>	El valor más bajo	Precio mínimo por categoría
<code>MAX(columna)</code>	El valor más alto	Venta máxima por empleado

Con `GROUP BY` y las funciones de agregación ya puedes resumir la operación completa de Tienda Latam en reportes concretos. El siguiente paso es aprender a filtrar esos resúmenes: no solo agrupar, sino quedarte solo con los grupos que cumplen una condición, como las categorías con más de diez productos o los meses con ventas superiores a cierto monto.